

Jenkins

Concepts:

1. Introduction to Jenkins
2. Installation of Jenkins
3. Creating first job
4. Git hub integration
5. Maven integration
6. Nexus integration
7. Tomcat integration
8. Sonar integration
9. User management
10. Email notification
11. Upstream and downstream
12. Pipeline
13. Master and slave set up
14. Jenkins Backup
15. Poll SCM, Webhook, Build Periodically

1. Introduction to Jenkins

Continuous Integration:

- > Whenever developers write a code, we integrate all the code of all developers at that point of time and we build, test and deliver/deploy to the client. This process is called CI/CD.
- > Jenkins helps in achieving this.
- > So instead of doing night builds as when commit occurs by integrating all code is what continues integration.
- > So, in a day, there will be so many integrations, builds, tests and delivers/deployments.
- > So that bugs will be repaired fast and get rectified fast. So development happens fast.
- > Your changes are getting integrated and quality checked continuously.
- > Simply it means every commit a developer does, immediately you integrate all the changes then do a build so that whether there is a problem or not you can find it out.
- > Commit all the changes in a repo then do a build.
- > Jenkins is an open source CI server - tool used to achieve CI.
- > Jenkins is maintained by Oracle.
- > Works in client-server model.
- > Dashboard shows the jobs, where they run and the status with history.

If new change is happened:

- A. Integrate (git pull)
- B. Repository (poll scm)
- C. Build (maven we will call)
 - a. Compile
 - b. Test

- c. Package
- d. Deploy
- D. Code coverage
- E. Code analysis

Key Terminology:

Integrate: Combine all the code written by developers till some point of time.

Build: Combine the code and make a small executable package.

Test: Test in all environments whether application is working properly or not.

Achieved: Stored in an artifactory so that in future we may use/deliver again.

Deliver: Handling the product to client.

Deploy: Installing product in client's machine.

Jenkins work flow:

- > We attach git, maven, and selenium and artifactory plugins to Jenkins.
- > Once developers put code in git, Jenkins pull that code and send to maven for build.
- > Once build is done, Jenkins pull that build code and send to selenium for testing.
- > Once testing is done, then Jenkins will pull that code and send to artifactory as per requirement.
- > We can also deploy with Jenkins.

Ways of continues Integration:

- > Can do manually
- > Can write scripts
- > Can use tool like Jenkins

Benefits of CI:

- > Defects caught early.
- > Delete bugs as soon as possible.
- > Maintains history for references.
- > If you want, you can stop SDLC process at any stage. For example, you can stop at test stage only or you can continue till deployment.
- > Easy to setup and maintain.
- > Easy integration with other tools by using plugins.
- > Can run on cloud infra as well as data center.

Why only Jenkins:

- > It has many plugins.
- > You can write your own plugin
- > You can use community plugins.
- > Jenkins is not a just tool. It is a framework means you can do anything whatever you want. All you need is plugins.

-> Jenkins is not going to do any job. It instructs others to do job. If other servers are not available, Jenkins itself does the job.

-> Jenkins also acts as a server replacement means you can do repeated tasks automatically.

-> Can create labels (group of servers)

Jenkins Architecture:

-> Client-server model.

-> When you install Jenkins, it acts as master.

-> Jenkins invoke clients to perform jobs.

-> Supported OS

-> Can install in any os.

-> You will be accessing Jenkins with web only.

-> choose long term support release.

-> install java

-> Open source- Jenkins

Enterprise edition- Hudson

-> Three types of configurations

- a. **Global system configuration:** Tools, environment variables.
- b. **Node configuration:** Job in which machine you want to run
- c. **Plugin Configuration:** In Jenkins master install plugins.
- d. **Job configuration:** All jobs will be configured here.

2. Installation & Configuration of Jenkins

Installation of Jenkins:

-> We have to ensure that our software packages are up to date on our instance by using the following command:

```
[ ] yum update -y
```

-> We have to add the Jenkins repo using the following command:

```
[ ] wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/rpm-stable/jenkins.repo
```

-> To Import a key file from Jenkins-CI to enable installation from the package:

```
[ ] rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
```

-> To upgrade

```
[ ] yum upgrade
```

-> To Install Java

```
[ ] yum install java-21-amazon-corretto -y
```

-> To Install Jenkins

```
[ ] yum install jenkins -y
```

-> To Enable the Jenkins service to start at boot:

```
[ ] systemctl enable jenkins
```

-> To Start Jenkins as a service:

```
[ ] systemctl start jenkins
```

-> We can check the status of the Jenkins service using the command:

```
[ ] systemctl status jenkins
```

Configuring Jenkins:

-> Connect to `http://<your_server_public_DNS>:8080` from the browser. We will be able to access Jenkins through its management interface:

```
[ ] http://PublicIP:8080
```

-> As prompted, enter the password found in `/var/lib/jenkins/secrets/initialAdminPassword`.

We have to use the following command to display this password:

```
[ ] cat /var/lib/jenkins/secrets/initialAdminPassword
```

-> The Jenkins installation script directs us to the Customize Jenkins page. We have to Click on Install suggested plugins.

-> Once the installation is complete, the Create First Admin User will open. We have to enter our information, and then select Save and Continue.

-> On the left-hand side, we have to select Manage Jenkins, and then we have to select Plugins.

-> Select the Available Plugins, and then enter Amazon EC2 plugin at the top right.

-> Select the checkbox next to Amazon EC2 plugin, and then select Install without restart.

-> Once the installation is done, select Back to Dashboard.

-> Select Configure a cloud if there are no existing nodes or clouds.

-> If we already have other nodes or clouds set up, select Manage Jenkins.

- a. After navigating to Manage Jenkins, select Configure Nodes and Clouds from the left hand side of the page.
- b. From here, select Configure Clouds.

-> Select Add a new cloud, and select Amazon EC2. A collection of new fields appears.

-> Click Add under Amazon EC2 Credentials

- a. From the Jenkins Credentials Provider, select AWS Credentials as the Kind.
- b. Scroll down and enter in the IAM User programmatic access keys with permissions to launch EC2 instances and select Add.
- c. Scroll down to select your region using the drop-down, and select Add for the EC2 Key Pair's Private Key.
- d. From the Jenkins Credentials Provider, select SSH Username with private key as the Kind and set the Username to `ec2-user`.
- e. Scroll down and select Enter Directly under Private Key, then select Add.
- f. Open the private key pair you created in the creating a key pair step and paste in the contents from `"-----BEGIN RSA PRIVATE KEY-----"` to `"-----END RSA PRIVATE KEY-----"`. Select Add when completed.
- g. Scroll down to "Test Connection" and ensure it states "Success". Select Save when done

-> We are now ready to use EC2 instances as Jenkins agents.

3. Creating first job in Jenkins:

- > Click on 'New Item,' then enter the item name, give it as a script, click on 'Freestyle project,' then click 'OK.'
- > Go to the Build tab, then click on Execute Shell. Click Apply and Save, then click Build Now. Finally, check the console output.
- > From CLI checking: `cd /var/lib/jenkins/jobs`

4. Git integration:

- > For git integration first we must install git in Jenkins server.

```
[ ] yum -y install git
```

- > Click on 'New Item,' then enter the item name, click on 'Freestyle project,' then click 'OK.'
- > Navigate to Source Code Management, click on Git, enter the URL, click Apply and Save, then click Build Now and check the console output.

5. Maven Integration

What is Maven?

Maven is a:

- Build tool
- Dependency management tool
- Documentation tool
- Project management tool
- Project lifecycle tool
- Build = compile + assembly + create deliverable
- Compile: Convert source code into machine-readable format.
- Assembly: Grouping all class files together.
- Deliverable: .ear, .war, .jar

Advantages of Maven:

- Automates tasks
- Runs multiple tasks at a time
- Produces a quality product
- Minimizes bad builds
- Keeps history
- Saves time
- Provides documentation

- Gives a set of standards
- Defines a project lifecycle
- Manages all dependencies
- Enables reusability
- > Why have a separate build team? To match the customer's environment.
- > Build tools by language:
 - C, C++ = Makefile
 - .NET = Visual Studio
 - Java = Ant, Maven

Maven Architecture:

- Maven's build system reads the project configuration (`pom.xml` / goals) and coordinates between your **Local Repository** and a **Remote Repository** (over HTTP) to fetch and store dependencies, ultimately producing outputs like a project site.
- Main configuration file: `pom.xml`
- Rule: one project → one workspace → one pom.xml

Requirements for a build:

- a. Source code (present in the workspace)
- b. Compiler (Maven compiler, not the Java compiler)
- c. Dependencies

Flow: Maven → Goals → Plugins → Task

- Anything you want to do in Maven is expressed in terms of goals.
- Goals invoke plugins.
- Plugins perform the actual task.
- > pom.xml lives inside the workspace, and goals are defined inside pom.xml. Whenever Maven runs, it goes to pom.xml and checks the goals specified there.

Inside pom.xml:

- List of sources → goals
- List of resources → plugins

Local repository check:

- In the current directory/local repo, Maven checks for the list of sources and resources. If resources are not available in the local repo, it checks the remote repo.

Maven Build Lifecycle. There are two types of lifecycles:

Default Lifecycle

- a) Generate sources and resources
- b) Compile (Java files converted into class files)
- c) Test (unit test)
- d) Package (.jar, .war, .ear)

- e) Install
- f) Deploy

Package (a.k.a. assembly) produces one deliverable:

- .jar (Java Archive) — contains one or more class files providing functionality.
- .war (Web Archive) — for web applications; contains web-related files needed to install a web application.
- .ear (Enterprise Archive) — also for web applications, but license-based.

Non-Default Lifecycle

Clean and site fall under the non-default lifecycle. Running a non-default lifecycle executes only that specific phase, not all phases.

- clean: Deletes all runtime files (.class files, test results). Does not delete source code.
- Site: An independent phase that generates project documentation — how the project is structured, what dependencies it has, etc.

Workspace / Project Directory Structure:

- a. src → src/main, src/test
- b. pom.xml → configuration file

Project Identity (GAV):

Maven uniquely identifies a project using GroupId, ArtifactId, Version (GAV):

- groupId: Arbitrary project grouping identifier
- artifactId: Arbitrary name of the project
- version: Version of the project

Syntax: groupId:artifactId:version

Maven Environment Setup:

- M2_HOME: Directory containing the contents of the Maven zip/tar file. Maven is installed under M2_HOME.
- M2: Refers to the bin folder under M2_HOME.

Creating a project:

```
[ ] mvn archetype:generate
```

Maven Repository:

- Remote Repository: Dependencies are downloaded from repositories via HTTP.
- Local Repository: Downloaded dependencies are cached locally.

POM (Project Object Model):

- POM stands for Project Object Model. It is the fundamental unit of work in Maven — an XML file containing information about the project and the configuration details Maven uses to build it.

POM contains:

- Name and version
- Artifact type

- Source code location
- Dependencies
- Plugins
- Profiles

Plugin Types:

- Build plugins: Default lifecycle phases are handled by build plugins.
- Reporting plugins: e.g., the site plugin.

Multi-Module Projects:

A project can be divided into modules. Each module acts as a separate mini-project.

Dependency Management:

- Dependencies are identified and referenced using GAV (GroupId, ArtifactId, Version).

Profiles:

- Profiles are groups of plugins that can be invoked together whenever needed.

Maven Snapshot vs. Version:

Version: If Maven has downloaded a dependency at version 1.0, it will not automatically download a newer 1.0. To get the latest version, the dependent project must upgrade to 1.1.

Snapshot: With a Maven snapshot, Maven automatically downloads the latest snapshot of the dependency project every time the project is built.

Maven Setup in Jenkins and Configuring the Maven tool:

- Go to Manage Jenkins → Global Tool Configuration → Maven
- Click Add Maven, give it a name and version
- Click Apply and Save

Example job configuration:

- Click New Item, give it a name, click OK
- Go to Configuration → Source Code Management → select Git → provide the repository URL
- Go to Build → Invoke top-level Maven targets
- Provide the Maven version and set the goal as install
- Click Apply and Save
- Click Build Now and check the console output

6. Nexus Integration

What is nexus?

- > Nexus is a repository manager, it stores "artifacts"
- > It allows you to collect, and manage your dependencies

-> It makes it easy to distribute your software. Internally, you configure your build to publish artifacts to Nexus and they then become available to other developers.

Types of repositories:

-> A repository group is a collection of other repositories, where you can combine multiple repositories of the same format into a single item.

-> A proxy repository is a repository that is linked to a remote repository. Any request for a component is verified against the local content of the proxy repository. If no local component is found, the request is forwarded to the remote repository.

-> This hosted repository can be used for third-party dependencies not available in the public Maven repositories.

Installing nexus:

```
[ ] yum install java-1.8.0-openjdk-devel -y
[ ] cd /opt
[ ] wget http://download.sonatype.com/nexus/3/nexus-3.15.2-01-unix.tar.gz
[ ] tar -zxvf nexus-3.15.2-01-unix.tar.gz
[ ] mv /opt/nexus-3.15.2-01 /opt/nexus
```

-> As a good security practice, Nexus is not advised to run nexus service as a root user, so create a new user called nexus and grant sudo access to manage nexus services as follows.

```
[ ] useradd nexus
```

-> Give the sudo access to nexus user

```
[ ] visudo
[ ] nexus ALL=(ALL) NOPASSWD: ALL
```

-> Change the owner and group permissions to /opt/nexus and /opt/sonatype-work directories.

```
[ ] chown -R nexus:nexus /opt/nexus
[ ] chown -R nexus:nexus /opt/sonatype-work
[ ] chmod -R 775 /opt/nexus
[ ] chmod -R 775 /opt/sonatype-work
```

-> Open /opt/nexus/bin/nexus.rc file and uncomment run_as_user parameter and set as nexus user.

```
[ ] vi /opt/nexus/bin/nexus.rc
[ ] run_as_user="nexus"
```

-> Create nexus as a service

```
[ ] ln -s /opt/nexus/bin/nexus /etc/init.d/nexus
```

-> Switch as a nexus user and start the nexus service as follows.

```
[ ] sudo su - nexus
```

-> Enable the nexus services

```
[ ] sudo systemctl enable nexus
```

-> Start the nexus service

```
[ ] sudo systemctl start nexus
```

-> Hit the browser with = PubIP:8081/nexus

-> Login with user name as admin and password is admin123

-> Click on repository and again click on hosted repository.

-> Give the repoid, name and click on save.

Configuring Job:

-> Go to Jenkins UI > install the nexus antifactory uploader plugin.

-> Go to Build and click on Nexus artifact uploader.

Parameters:

-> Nexus URL: PubIP:8081/nexus

-> Credentials: username as admin and password is admin123

Groupid: vprofile-v1

Version: \$BUILD_ID

Repository: Repoid

Artifactid: vprofile-v1

Type: war

File: target/vprofile-v1.war

Checking Output:

-> Go to Nexus UI and check the output

7. Tomcat integration

Continuous Deployment:

-> CD is the practice to continuously release software in production.

-> Code changes are continuously built, tested and pushed to a non production and production environments by using automatin tools.

-> Software delivery cycles are more rapid and effective.

Advantages of Continuous deployment:

-> Automates software releases

-> Increase developer productivity.

-> Locates and addressing bugs quicker

Continuous Delivery:

-> Continuous Delivery is a process, where code changes are automatically built, tested, and prepared for a release to production.

-> Continuous Delivery aims to make software releases, remove risk, so we can deliver frequently and get fast feedback on what users care about.

-> Manually giving product to client and we are not deploying product in application servers.

Advantages of Continuous Delivery:

-> Automates software releases

-> Increase developer productivity.

-> Locates and addressing bugs quicker.

Tomcat for Continuous deployment:

-> Apache tomcat is used as application server where we are deploying our code.

-> Application servers are used to deploy the code

-> JBoss, tomcat used as application servers.

Tomcat Installation:

```
[] yum install java-1.8.0-openjdk-devel -y
```

```
[] wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.120/bin/apache-tomcat-9.0.120.tar.gz
```

```
[] tar -xvzf apache-tomcat-9.0.120.tar.gz
```

```
[] cd apache-tomcat-9.0.120
```

```
[] ls -latr
```

```
[] cd conf/
```

-> Add below lines in tomcatusers.xml

```
[] vi tomcatusers.xml
```

```
<role rolename="manager-gui"/>
```

```
<role rolename="manager-script"/>
```

```
<user username="tomcat" password="tomcat" roles="manager-script,manager-gui"/>
```

-> After adding above lines save the file

-> Navigate to apache-tomcat-9.0.120/webapps/manager/META-INF

```
[] vi context.xml
```

-> Comment the below line

```
<!--
```

```
<Valve className="org.apache.catalina.valves.RemoteAddrValve"
```

```
allow="127\.\d+\.\d+\.\d+|::1|0:0:0:0:0:0:0:1" />
```

```
-->
```

-> Save the file and exit.

-> Start the tomcat

```
[] cd apache-tomcat-9.0.120/bin
```

```
[] ./startup.sh
```

- > Here tomcat will start
- > Hit the browser with PubIP:8080
- > Click on manager app and login with credentials.

8. Sonar integration

- > Once we develop the code, we need to check the quality of product.

Test case:

- > A test case is usually a sequence of steps, to test behavior, functionality, feature of an application.

Types of testing:

- > Unit test: pieces of code can be tested.
- > Integration test: Complete product will be tested.
- > Smoke testing: Product will be given by development team to QA. Code will be testing in QA.
- > System testing: It can be done by QA. They will check it will be working functionally fully or not.
- > Regression Testing: Any modification is added to the existing application, test the complete product.
- > Performance testing: To check access speed, logging many users, effectiveness of system, load, how much time it is taking to load a page.
- > Beta testing: It is done by end users. It is used to find unexpected errors.

Code Coverage:

- > Example a product developed, product contains 1000 lines. When you try to execute product, we sure all 1000 lines are executed or not.
- > Developers may write unwanted code. So we are checking all 1000 lines are using or not. We are verifying.
- > We are checking, how many lines of code we are checked, how many lines of code need for product.
- > It is measurement of how many lines are executed when application is running.
- > When you eliminate unwanted lines, the quality of product is higher.
- > We can improve quality of product by using code coverage.
- > It is part of build management.
- > Code coverage details: functional coverage, standard coverage, Branch coverage, condition coverage.
- > Code coverage metrics: It is good to achieve 100%. But 90% is an ideal percentage.

Tools: Sonarqube

Sonarqube etc

- > code quality and code analysis we can perform by using sonar qube.

Sonarqube scanner:

- > Sonarqube scanner which comes with sonarqube tool.

- > Sonarqube is a web application for displaying output.
- > Sonarqube scanner which will be running against for our project.
- > Sonarqube scanner running for code analysis.
- > sonar qube should install where you are doing the build.

Sonar installation:

Prerequisites:

- > ubuntu- AMI ID

```
[] ubuntu/images/hvm-ssd/ubuntu-bionic-18.04-amd64-server-20200908 (ami-0817d428a6fb68645)
```

- > Medium instance

Installing Java:

```
[] apt-get update
>[] apt install openjdk-11-jdk
>[] java -version
```

Installing sonar:

```
[] wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-9.9.3.79811.zip
>[] apt install unzip -y
>[] unzip sonarqube-9.9.3.79811.zip
>[] mv sonarqube-9.9.3.79811 /sonarqube
```

Installing MySQL:

```
[] apt install mysql-server -y
>[] systemctl start mysql
```

Login to the database:

```
[] mysql -u root -p
>[] pwd: root
```

- > Create the database

```
[] MySQL > CREATE DATABASE sonar CHARACTER SET utf8mb4 COLLATE utf8mb4_bin;
```

- > Create the user (This covers connections from localhost/local machine)

```
[] MySQL > CREATE USER 'sonar'@'localhost' IDENTIFIED BY 'sonar';
```

- > Create the user for remote connections (If needed for Docker/other servers)

```
[] MySQL > CREATE USER 'sonar'@'%' IDENTIFIED BY 'sonar';
```

- > Grant privileges to the local user

```
[] MySQL > GRANT ALL PRIVILEGES ON sonar.* TO 'sonar'@'localhost';
```

- > Grant privileges to the remote user

```
[] MySQL > GRANT ALL PRIVILEGES ON sonar.* TO 'sonar'@'%';
```

-> Apply the changes

```
[] MySQL > FLUSH PRIVILEGES;
```

```
[] MySQL > exit
```

Provide sonar user name and password in sonar properties file:

```
[] cd sonarqube-9.9.3.79811
```

```
[] cd conf
```

```
[] vi sonar.properties
```

-> Please make sure below changes should be done.

```
> sonar.jdbc.username=sonar
```

```
> sonar.jdbc.password=sonar
```

```
>
```

```
sonar.jdbc.url=jdbc:mysql://localhost:3306/sonar?useUnicode=true&characterEncoding=utf8&rewriteBatchedStatements=true&useConfigs=maxPerformance
```

```
> sonar.web.host=0.0.0.0
```

```
> sonar.web.port=9000
```

Start the sonar:

```
[] cd sonarqube-9.9.3.79811
```

```
[] cd bin
```

```
[] cd linux-x86-64
```

```
[] ./sonar.sh start
```

```
[] ./sonar.sh status
```

Run as a service:

-> Create a systemd service so SonarQube starts at boot:

```
[] cd /etc/systemd/system
```

```
[] vi sonarqube.service
```

```
[Unit]
```

```
Description=SonarQube service
```

```
After=network.target mysql.service
```

```
Wants=mysql.service
```

```
[Service]
```

```
Type=forking
```

```
User=root
```

```
Group=root
```

```
ExecStart=/sonarqube/bin/linux-x86-64/sonar.sh start
```

```
ExecStop=/sonarqube/bin/linux-x86-64/sonar.sh stop
```

```
Restart=always
```

```
RestartSec=10
```

```
LimitNOFILE=65536
```

```
LimitNPROC=4096
```

```
TimeoutStartSec=300
```

```
TimeoutStopSec=60
```

```
[Install]
```

```
WantedBy=multi-user.target``
```

-> Then enable it

```
[] systemctl daemon-reload
```

```
[] systemctl enable --now sonarqube
```

-> Hit the SonarQube server IP at browser

Ex: PubIP:9000

-> Default login is usually:

```
[] Username: `admin`
```

```
[] Password: `admin`
```

9. User management

-> We can provide access to different users in Jenkins and also can provide them with roles based on project-based access.

-> Go to Manage Jenkins > Manage Users > Create user > Set Credentials

-> Go to Jenkins > configure global security > Authorization > Matrix based authentication > click on add user > apply and save.

10. Email notification

-> Jenkins comes with an out of box facility to add an email notification for a build project.

-> The use of email notification is that we will get to know particular build is success or failed.

-> Here's a simple step-by-step guide to set up Jenkins email notifications:

Step 1: Install the Plugin

-> Log into Jenkins > Go to Manage Jenkins > Plugins > Click on Available plugins > Search for Email Extension Plugin > Click on Install

Step 2: Get Your Email Ready (Gmail example)

-> If you're using Gmail, you need an App Password (not your normal password):

-> Go to your Google Account > Security > Turn on 2-Step Verification (if not already on) > Go to App Passwords > Generate a new one for "Mail" > Copy the 16-character password — you'll need it in Step 3

Step 3: Configure SMTP in Jenkins

-> Go to Manage Jenkins > System > Scroll to Extended E-mail Notification

-> Example Gmail SMTP config:

☐ SMTP server: smtp.gmail.com

☐ Port: 587

☐ Use SMTP Authentication: yes

☐ Username: name@gmail.com

☐ Password: app-password (not a regular password)

☐ Use TLS: yes

-> Scroll down, click Test configuration by sending test e-mail > Enter your email > Click Test > Check your inbox — if you got the email, it works!

Step 4: Add Email Alerts to Your Job

If you use Freestyle Jobs:

-> Open your Job > Configure > Scroll to Post-build Actions > Click Add post-build action > Editable Email Notification > Enter recipient email(s) > Save

Step 5: Run the Job and Check

-> Trigger a build (manually or via commit)

-> Wait for it to finish

-> Check your email — you should get a notification based on success/failure

11. Upstream and Downstream

-> The Upstream job is the one that is triggered before the actual job is triggered.

-> The Downstream job is the one that is triggered after the actual job is triggered.

Setting downstream:

-> Create new item and go to configure.

-> Go to post build actions and click on other projects to build

-> Click on apply and save.

12. Pipeline

-> Jenkins pipeline is collection of jenkins jobs which trigger eachother in a specefic sequence.

-> We can create pipeline in 2 ways.

a. Pipeline view

-> When you have complex builds Pipelines, it is useful to be able to see the progress of each stage.

-> This approach is good when you are having less no. of jobs.

Configure Pipeline view:

b. Pipeline Script or jenkins pipeline:

-> Jenkins pipeline is a single platform that runs entire pipeline as a code.

-> Instead of building several jobs for each phase, you can code entire work flow and put in a jenkins file.

Key features of jenkins pipeline:

-> Pipeline as code

-> Code can be stored in git. (Developers can easily access and edit the code at any point of time)

-> Integrates with other plugins like git, maven, tmcat etc.

Ways to writing the jenkins pipeline:

a. Declarative pipeline:

-> Recent feature.

-> Simple groovy syntax

-> The code is defined within a pipeline block.

b. Scripted pipeline

-> Traditional way of writing code.

-> Using groovy syntax

-> Code is written on the jenkins UI interface.

Differences between scripted pipeline and declarative pipeline:

-> The key difference between Declarative pipeline and Scripted pipeline would be with respect to their syntaxes and their flexibility.

-> Unlike Declarative pipeline, the scripted pipeline strictly uses groovy based syntax

-> Declarative Pipeline encourages a declarative programming model, whereas Scripted Pipelines follow a more imperative programming model.

Example of scripted pipeline:

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
    }
}
```

```

stage('Test') {
    steps {
        sh 'mvn test'
    }
}

stage('Docker Build') {
    steps {
        sh 'docker build -t java-jenkins-docker:latest .'
    }
}

stage('Docker Run') {
    steps {
        sh 'docker run --rm java-jenkins-docker:latest'
    }
}
}

```

13. Master and Slave setup

-> To distribute the workload we are creating slaves.

-> Jenkins supports the master-slave architecture

-> Many slaves work for a master. It is also known as Jenkins Distributed Builds.

Node setup:

-> Install java in slave

```
[] yum install java-1.8.0-openjdk-devel
```

-> Create user and assign password to user

```
[] useradd slave
```

```
[] passwd slave
```

-> Make the changes in configuration file

```
[] cd /etc/ssh/ssd_config
```

```
> permistEmptypasswords no
```

```
> challengeresponseauthentication no
```

```
> paswordauthentication yes
```

-> Restart sshd service

```
[] systemctl start sshd
```

-> Give sudo access to slave

```
[] visudo
```

```
> slave  ALL=(ALL)  ALL
```

-> Switch to slave user

```
[] su - slave
```

-> Create directory

```
[] mkdir jenkins-home
```

```
[] cd jenkins-home
```

-> Now hit the jenkins at your browser

Creating node In Jenkins:

-> Go to Manage Jenkins > Manage node > Create New Node

> Node Name: Whatever I want

> No. of Executors: As much as I want

> Remote root directory: /Dir-Name

> Labels: Slave123 (Label is nothing but a Group)

> Save

-> Node is created but not active so we have to configure it. Click on Name of the node

-> Run commands from agent

```
[] echo ab93f8ca5719905b4d74a0e1f085bb74f8ccfca6d97cad60a5fad50376965d81 > secret-file
```

```
[] curl -v -O http://54.237.91.162:8080/jnlpJars/agent.jar
```

```
[] java -jar agent.jar -url http://54.237.91.162:8080/ -secret @secret-file -name Slave -webSocket -workDir
"/jenkins-home"
```

-> Create new job > Click on Restrict where this project can be run > Give Label > Click on GIT > Add GIT Repo > Apply and save > Build Now

1. Jenkins backup

Introduction:

-> Jenkins has a backup plugin which can be used to backup critical configurations settings related to Jenkins.

-> Install Backup plugin: Backup plugin

Setup:

-> Navigate to manage Jenkins > Backup manager > click on backup manager > Click on setup

-> We have to mention the directory name where we want to take backup.

-> Click on Backup Hudson configuration

-> In this step Jenkins is taking backup and before taking backup Jenkins is shutting down.

15. Differences between build periodically, pollscm and Webhook:

Build periodically:

-> Build periodically builds the project periodically even if nothing has committed in the code.

Pollscm:

-> Poll scm checks whether changes were made and builds the project if new commits are there.

-> For example polling frequency is every 12 hours and just now poll happened. Very next minute if the code got committed in the git master branch but the build will not happened until the next poll. It will wait for the next poll.

Webhook:

-> Used to trigger the builds whenever a developer commits to the master branch.

How to setup webhook:

-> Go to your github account > go to your repo > settings > click on webhook

-> Here select only pull requests and pushes only

-> Click on active and add webhhok.

-> Open jenkins console add git URL under source code management.

-> Go to build triggers > select the GitHub hook trigger for GITScm polling

-> Click on apply and save.